

Measuring the Adoption of SSCSP in Public Software Repositories

Marius Spill, 200686670

Mentor: Dominik Wermke

Spring 2026

Abstract

The P-SSCRM framework provides a structured approach for mitigating software supply chain risks through the adoption of defined security practices. While prior work has focused on the design and conceptual integration of these practices, empirical evidence on their adoption in public open-source software remains limited. This study investigates the extent to which selected P-SSCRM practices are adopted in large, widely used open-source repositories on GitHub.

A sample of 30 high-impact repositories, primarily representing infrastructure and widely used libraries, was analyzed. Two specific practices were examined: (1) the avoidance of binary artifacts within repositories and (2) the presence of multi-party (two-party) code review processes. Repository contents and pull request activity were collected via the GitHub API and analyzed using a structured data pipeline and SQL-based evaluation.

The results indicate that binary artifacts are largely avoided across most repositories, while consistent two-party review enforcement varies significantly between projects. These findings provide initial empirical evidence on the uneven adoption of software supply chain security practices in open-source ecosystems and highlight areas where further standardization and enforcement may be required.

Problem Statement

Software supply chain security has become a critical concern due to the increasing frequency and impact of attacks targeting dependencies, build systems, and distribution pipelines. Frameworks such as the P-SSCRM framework define a wide range of practices intended to mitigate these risks, including multi-party code review and strict control over repository artifacts.

However, these frameworks are largely derived from industry settings and provide limited insight into how widely such practices are adopted in public open-source ecosystems. In particular, there is a lack of empirical measurement of security practice adoption in repositories hosted on

platforms such as GitHub. This gap makes it difficult to assess the current state of software supply chain security in open-source projects and to evaluate how well existing frameworks translate to this context.

To address this limitation, this study focuses on empirically measuring the adoption of selected software supply chain security practices in large public open-source repositories.

Literature Summary

Software supply chain security has gained increasing attention due to the rise of large-scale attacks targeting dependencies, build systems, and distribution channels. Incidents such as Log4j and SolarWinds demonstrate how vulnerabilities in widely used components can propagate across entire ecosystems, making the software supply chain a critical attack surface.

To address these risks, the P-SSCRM framework was introduced as a unified model for software supply chain risk management. The framework is based on the analysis of real-world industry initiatives and integrates ten existing standards and frameworks, including NIST SSDF, SLSA, and OWASP SCVS . It defines 73 tasks organized into practices and groups, providing a structured way to describe and compare software supply chain security efforts.

However, the P-SSCRM framework is primarily descriptive and focuses on practices observed in industry settings rather than measuring their adoption in open-source ecosystems . As a result, there is limited empirical evidence on how widely these practices are implemented in public repositories, particularly on platforms such as GitHub.

This gap motivates the present study, which aims to empirically assess the adoption of selected P-SSCRM practices in large open-source projects.

Methodology / Process

This study employs an empirical, data-driven approach to measure the adoption of selected software supply chain security practices in public open-source repositories. The methodology consists of repository selection, data collection via the GitHub API, structured data storage, and metric-based analysis.

Repository Selection

A sample of 30 large and widely used open-source repositories was selected from GitHub. The selection focused on infrastructure projects, libraries, and core technologies that are actively maintained and broadly used in other software systems. Repositories that primarily serve as mirrors of external version control systems were excluded to ensure that development activity is accurately represented.

Data Collection

Data was collected using the GitHub REST API. For each repository, metadata and historical development activity were retrieved, including pull requests, comments, and review data. Due to inconsistencies and incomplete availability of review data across repositories, pull request comments were additionally collected and used as a proxy for code review activity.

To focus the analysis on recent and relevant development practices, a sampling strategy was applied. For each repository, the most recent pull requests were identified, and a fixed number of the newest comments and reviews (approximately 500 per repository) were retrieved in batches. This approach ensures comparability across repositories while maintaining a manageable dataset size.

Data Storage and Processing

All collected data was stored in a structured SQLite database. The database schema includes separate tables for repositories, pull requests, comments, and reviews. Data ingestion was implemented with idempotent insert operations to ensure consistency across repeated API calls.

For binary artifact analysis, repository file structures were traversed using the GitHub API. Files were classified into categories such as binary files, compiled artifacts, and archive files based on file extensions and metadata. Aggregated statistics were computed for each repository and stored in a derived summary table.

Operationalization of Security Practices

Two software supply chain security practices were operationalized as measurable metrics:

Multi-party Code Review:

Due to incomplete review data, multi-party review was approximated using pull request comments. For each pull request, distinct non-author and non-automated commenters were identified. A pull request was classified as having multi-party review if at least two independent contributors (excluding the author) participated in the discussion. This serves as a proxy for two-party review processes.

Avoidance of Binary Artifacts:

The presence of binary artifacts was measured by analyzing repository contents. For each repository, the ratio of binary files, compiled artifacts, and archive files relative to the total

number of files was computed. Additionally, a binary indicator was used to classify repositories that strictly avoid binary artifacts.

Data Analysis

The analysis was conducted using SQL-based queries on the constructed dataset. Metrics were aggregated at the repository level and compared across projects. The study examines the prevalence of the selected practices and explores variation across repositories, providing an initial empirical assessment of software supply chain security practice adoption in open-source ecosystems.

Results / Contribution

Multi-Party Code Review Analysis

One commonly recommended software supply chain security practice is the use of multi-party code review. The core idea is that changes should not be accepted solely by their original author, but instead receive oversight or participation from additional contributors. This reduces the risk of accidental defects, insecure changes, or malicious contributions being merged without scrutiny.

Operational Definition

Direct measurement of formal branch protection rules or mandatory reviewer requirements is not consistently available across historical GitHub repository data. Therefore, this study uses an observable behavioral proxy based on pull request activity.

A pull request was classified as exhibiting **multi-party review** when at least one distinct non-author human participant was observed interacting with the pull request through review records or review-related comments. Automated accounts and known bots were excluded where identifiable. The pull request author was also excluded from the participant count.

Under this definition:

- **Pull request author alone** = not multi-party review
- **Author + at least one additional human reviewer/commenter** = multi-party review

This proxy does not claim that a repository formally enforces two-person approval. Rather, it measures whether review activity involving multiple contributors is visibly present in sampled pull requests.

Measurement Challenges

This practice is substantially harder to measure than it may initially appear. Public GitHub data creates several limitations:

1. **Repository workflows differ widely**

Some projects rely heavily on GitHub review approvals, while others use issue discussions, mailing lists, external code review systems, or maintainer workflows not fully visible through GitHub pull request metadata.

2. **Historical API inconsistencies**

Review records were not equally complete across repositories. Some large projects returned sparse review objects despite clearly active contribution processes.

3. **Bot activity and automation noise**

Many repositories use bots for CI checks, dependency updates, merge assistance, or release automation. These accounts can appear in pull request discussions and had to be filtered where possible.

4. **Comments do not always imply substantive review**

A comment may represent a meaningful code review, but it may also be administrative, procedural, or minor clarification.

5. **Absence of evidence is not evidence of absence**

If no reviewer was observed, this does not necessarily mean no review occurred.

For these reasons, the metric should be interpreted as an estimate of observable collaborative review behavior rather than a definitive compliance score.

Results

Across the analyzed repositories, substantial variation was observed in estimated multi-party review adoption.

Projects with comparatively high observed rates included:

- rust-lang/rust (~53.6%)
- apache/airflow (~53.6%)
- kubernetes/kubernetes (~53.4%)
- nodejs/node (~48.6%)
- jenkinsci/jenkins (~46.0%)
- llvm/llvm-project (~44.3%)

These repositories showed frequent evidence that pull requests involved at least one additional human participant beyond the author.

A middle tier of repositories showed moderate rates, generally between approximately 20% and 40%, including projects such as Redis, Keras, Helm, Apache Spark, PyTorch, Prometheus, Grafana, Bazel, Docker CLI, and etcd.

Several repositories showed comparatively low observed rates, including:

- facebook/react (~9.1%)
- hashicorp/terraform (~7.7%)
- sigstore/sigstore (~7.3%)
- anchore/grype (~5.9%)
- golang/go (~0.6%)

These low values should be interpreted carefully, as they may reflect workflow characteristics or data visibility limitations rather than weak security practices.

Interpretation

The results suggest that collaborative review practices are present across many mature open-source projects, but the visible form of those practices differs considerably. Larger infrastructure-oriented projects with extensive governance structures often showed stronger evidence of multi-party review participation. Other repositories may rely on alternative governance models, smaller trusted maintainer groups, or review processes that are less visible in GitHub pull request metadata.

Therefore, this metric is most useful for **relative comparison** and ecosystem trend analysis rather than strict pass/fail judgments.

Conclusion

Multi-party review appears to be meaningfully adopted in many prominent public open-source repositories, but adoption is heterogeneous and difficult to measure perfectly using public platform data alone. The findings support the broader conclusion that supply chain security practices do exist in open-source ecosystems, though often implemented through repository-specific processes rather than uniform formal controls.

Binary Artifacts in Repositories

Another practice examined in this study is the avoidance of binary artifacts in source repositories. This practice is relevant to software supply chain security because binary files are generally harder to inspect, review, and reproduce than plain-text source files. In particular, compiled binaries and packaged archives can obscure what is actually being included in a repository and may bypass normal source-level review processes. Keeping repositories focused on human-readable source artifacts is therefore often considered a more secure and auditable practice.

Operational Definition

To measure this practice, each repository's default branch was scanned at the file-tree level. All files were classified by extension into one of several categories.

The analysis treated the following extensions as **binary assets**:

- `.png, .jpg, .jpeg, .gif, .svg, .pdf`

These files are binary in a technical sense, but they are often legitimate repository contents such as images, diagrams, or documentation assets.

The analysis treated the following extensions as **compiled or suspicious binary artifacts**:

- `.exe, .dll, .so, .dylib, .jar, .class, .bin, .o, .a`

These are more directly relevant from a supply chain security perspective, since they may represent compiled objects, libraries, or executables that are not easily auditable in the same way as source code.

The analysis also classified the following as **archives**:

- `.zip, .tar, .gz, .tgz, .7z, .rar`

For each repository, the scan recorded:

- total number of files
- number of binary files overall
- number of compiled/suspicious binary files
- number of archive files
- whether the repository contained any compiled/suspicious binary artifacts (`has_strict_binary`)

In the results table, three ratios were computed:

- **binary_ratio**: all detected binary files divided by total files
- **compiled_binary_ratio**: compiled/suspicious binary files divided by total files
- **archive_ratio**: archive files divided by total files

The variable **has_strict_binary** indicates whether at least one compiled/suspicious binary artifact was present.

Measurement Challenges

This measure is more straightforward than the multi-party review metric, but it still has important limitations.

First, the method relies entirely on **file extensions**. This makes the analysis scalable and reproducible, but it is necessarily imperfect. A file with a suspicious extension may be harmless in context, while a file without such an extension may still be binary. The metric therefore captures a practical approximation rather than a complete ground-truth inventory.

Second, not all binary files are equally problematic. Many repositories legitimately contain images, diagrams, logos, PDFs, or other documentation assets. A repository can therefore have a relatively high overall binary ratio without necessarily violating the spirit of source-centric development. This is why the distinction between general binary files and stricter compiled/suspicious artifacts is important.

Third, repository structure varies substantially across projects. Some projects include test fixtures, packaged examples, vendored dependencies, generated artifacts, frontend assets, or release-related materials inside the repository. These choices can inflate binary counts without necessarily indicating poor security hygiene.

Fourth, this measurement is a **snapshot** of the default branch at scan time. It does not capture historical changes, deleted artifacts, or binary material that may exist only in releases rather than in the repository tree itself.

For these reasons, the results should be interpreted carefully. The metric is useful for comparing repository contents at a high level, but not for making simplistic pass/fail judgments.

Results

The results show substantial variation across repositories.

At the high end of the overall binary ratio, several repositories contain a comparatively large share of binary files:

- [mrdoob/three.js](#) (0.196)
- [scikit-learn/scikit-learn](#) (0.108)
- [grafana/grafana](#) (0.087)
- [helm/helm](#) (0.060)
- [apache/airflow](#) (0.056)

These repositories appear to include a notable amount of non-source binary material, such as images, documentation assets, packaged resources, or other binary content.

A middle group includes projects such as:

- [apache/spark](#) (0.040)
- [jenkinsci/jenkins](#) (0.035)
- [angular/angular](#) (0.032)

- [llvm/llvm-project](#) (0.025)
- [etcd-io/etcd](#) (0.024)
- [bazelbuild/bazel](#) (0.022)
- [vercel/next.js](#) (0.021)

These projects contain some measurable binary content, but not at especially extreme levels.

At the low end, several repositories show very small binary shares:

- [rust-lang/rust](#) (0.001)
- [kubernetes/kubernetes](#) (0.001)
- [redis/redis](#) (0.002)
- [nodejs/node](#) (0.002)
- [docker/cli](#) (0.002)

Finally, two repositories in the sample showed no detected binary files under this method:

- [keras-team/keras](#)
- [sigstore/sigstore](#)

Strict Binary Interpretation

The stricter indicator, [has_strict_binary](#), is more important from a supply chain security perspective than the overall binary ratio. This flag focuses on compiled or suspicious binary artifacts rather than ordinary documentation assets.

Under this stricter measure, repositories such as the following were flagged as containing potentially more concerning binary content:

- [mrdoob/three.js](#)
- [apache/spark](#)
- [jenkinsci/jenkins](#)
- [llvm/llvm-project](#)
- [bazelbuild/bazel](#)
- [vercel/next.js](#)
- [aquasecurity/trivy](#)
- [cockroachdb/cockroach](#)
- [golang/go](#)
- [tensorflow/tensorflow](#)
- [hashicorp/terraform](#)
- [pytorch/pytorch](#)
- [moby/moby](#)

- `anchore/grype`
- `nodejs/node`
- `rust-lang/rust`

By contrast, repositories such as `kubernetes/kubernetes`, `redis/redis`, `prometheus/prometheus`, `etcd-io/etcd`, `facebook/react`, `grafana/grafana`, `scikit-learn/scikit-learn`, and `sigstore/sigstore` were not flagged under this stricter criterion.

This distinction matters because a repository may contain many binary assets while still avoiding compiled executables, libraries, or packaged binaries. For example, a project with many images or PDFs may have a relatively high `binary_ratio` but still not contain strict binary artifacts.

Interpretation

The results suggest that avoidance of binary content is not uniform across prominent open-source repositories. Some projects remain highly source-centric, with almost no detectable binary material, while others include substantial amounts of binary assets as part of their normal repository structure.

However, the most meaningful interpretation comes from separating **general binary content** from **compiled or suspicious binary artifacts**. General binary content often reflects documentation, frontend resources, or test data and is not automatically a security concern. The stricter binary flag is therefore the more relevant indicator when interpreting repository practices in a software supply chain security context.

Even so, the presence of strict binary artifacts should not automatically be interpreted as poor practice. In some cases, these files may serve legitimate development, testing, or build-related purposes. The metric should therefore be viewed as a useful signal of repository content choices rather than a definitive judgment of security quality.

Conclusion

The repository scans show that binary artifact usage varies considerably across public open-source projects. Some repositories contain virtually no binary material, while others include substantial binary content. This heterogeneity reflects the fact that open-source repositories serve different technical purposes and follow different organizational conventions.

From a software supply chain security perspective, the stricter presence of compiled or suspicious binary artifacts is the most informative result. It provides an indication of where repositories include content that is less transparent and less easily reviewed than ordinary source files. At the same time, the method's dependence on extension-based classification

means that these findings should be interpreted as approximate indicators rather than exact compliance assessments.

Citations

References

Boote, J., Hutchison, B., Miguez, S., & Williams, L. (2025). *Proactive Software Supply Chain Risk Management Framework (P-SSCRM) (Version 1.01)*. North Carolina State University, Imbricate Security, and Black Duck.

Source Note

The P-SSCRM framework was the only external source used to guide the conceptual selection of software supply chain security practices examined in this study. All repository data collection, metric design, database construction, empirical analysis, and result interpretation were conducted independently by the author using publicly available GitHub data.

Generative AI tools (ChatGPT) were used as a productivity aid for coding assistance, structuring text, debugging support, and language refinement. Final methodological decisions, measurements, analysis design, and conclusions remained the responsibility of the author.

Reflection Elements

This project involved a number of practical and methodological challenges that are common in empirical software engineering research.

One major difficulty was understanding the available data sources, APIs, and supporting technologies well enough to build a reliable data collection pipeline. Although GitHub provides extensive public data, transforming raw API responses into a clean and analyzable dataset required substantial effort. Repository structures, pull request metadata, review events, comments, and branch information all had to be interpreted correctly before meaningful analysis could begin. In addition, working with Python, SQLite, automated scripts, and rate-limited web APIs required continuous debugging and iteration throughout the project.

A second challenge was data quality. Public platform data is not collected for research purposes and therefore contains noise, inconsistencies, and ambiguities. For example, automated accounts frequently appeared in review or comment activity and had to be filtered manually

where possible. Distinguishing between genuine human review participation and bot-generated interactions was necessary for the multi-party review metric, but not always straightforward. Similarly, repository contents often required interpretation, since binary files can serve many legitimate purposes and are not inherently problematic.

A third challenge was obtaining a sufficiently large and diverse sample size. Meaningful comparisons across open-source repositories require more than a small number of projects or isolated observations. Collecting data across many repositories with substantial pull request histories required careful batching, checkpointing, and repeated extraction runs due to API limits and repository scale. Building a sample large enough to support comparative analysis was therefore a significant part of the project itself.

Despite these challenges, the project provided valuable experience in empirical research methods. It demonstrated the importance of operationalizing abstract concepts into measurable variables, recognizing threats to validity, and interpreting results cautiously rather than assuming raw numbers speak for themselves. It also highlighted the tradeoff between measurement precision and scalability: simple proxies can enable large-sample analysis, but they must be discussed transparently.

Overall, the project provided practical experience in data collection, cleaning, reproducible analysis, and evidence-based reasoning. It also showed that empirical software engineering often depends as much on careful methodology and data handling as on technical implementation itself.

Appendix

Multiple Party Review Result Table

repo_id	repo_full_name	pr_count	observed_prs	multi_party_prs	coverage_rate	multi_party_rate
724712	rust-lang/rust	90601	834 ▾	447	0.009 ▾	0.536 ▾
33884891	apache/airflow	42565	825 ▾	442	0.019 ▾	0.536 ▾
20580498	kubernetes/kubernetes	87809	858 ▾	458	0.01 ▾	0.534 ▾

27193779	nodejs/node	39315	838 ▾	407	0.021 ▾	0.486 ▾
1103607	jenkinsci/jenkins	11644	574 ▾	264	0.049 ▾	0.460 ▾
75821432	llvm/llvm-project	87806	3038 ▾	1347	0.035 ▾	0.443 ▾
74175805	istio/istio	36211	607 ▾	261	0.017 ▾	0.430 ▾
24195339	angular/angular	34764	882 ▾	356	0.025 ▾	0.404 ▾
156018	redis/redis	7479	738 ▾	289	0.099 ▾	0.392 ▾
33015583	keras-team/keras	8733	861 ▾	332	0.099 ▾	0.386 ▾
43723161	helm/helm	6485	587 ▾	211	0.091 ▾	0.359 ▾
17165658	apache/spark	54237	864 ▾	310	0.016 ▾	0.359 ▾
65600975	pytorch/pytorch	117512	1383 ▾	433	0.012 ▾	0.313 ▾
6838921	prometheus/prometheus	11025	657 ▾	198	0.060 ▾	0.301 ▾
7691631	moby/moby	27887	548 ▾	154	0.020 ▾	0.281 ▾
15111821	grafana/grafana	72010	982 ▾	272	0.014 ▾	0.277 ▾
180687624	aquasecurity/trivy	4360	612 ▾	167	0.140 ▾	0.273 ▾
45717250	tensorflow/tensorflow	66892	104 ▾	28	0.002 ▾	0.269 ▾
843222	scikit-learn/scikit-learn	20188	786 ▾	204	0.039 ▾	0.260 ▾

20773773	bazelbuild/ bazel	11268	607 ▾	148	0.054 ▾	0.244 ▾
88013053	docker/cli	4813	866 ▾	200	0.180 ▾	0.231 ▾
11225014	etcd-io/etc d	13607	872 ▾	190	0.064 ▾	0.218 ▾
16563587	cockroach db/cockroa ch	81578	890 ▾	158	0.011 ▾	0.178 ▾
576201	mrdoob/thr ee.js	19287	646 ▾	93	0.033 ▾	0.144 ▾
70107786	vercel/next .js	34632	847 ▾	116	0.024 ▾	0.137 ▾
10270250	facebook/r eact	18986	696 ▾	63	0.037 ▾	0.091 ▾
17728164	hashicorp/t erraform	16083	882 ▾	68	0.055 ▾	0.077 ▾
338633268	sigstore/si gstore	2135	411 ▾	30	0.193 ▾	0.073 ▾
267054247	anchore/gr ype	2069	353 ▾	21	0.171 ▾	0.059 ▾
23096959	golang/go	4863	792 ▾	5	0.163 ▾	0.006 ▾

Binary Artifacts results

repo_id	repo_full_ name	total_files	binary_rat io	compiled_ binary_rat io	archive_ra tio	has_strict _binary
576201	mrdoob/thr ee.js	5886	0.196 ▾	0.003 ▾	0.001 ▾	1 ▾
843222	scikit-learn	1799	0.108 ▾	0.0 ▾	0.046 ▾	0 ▾

	/scikit-learn					
15111821	grafana/grafana	21454	0.087 ▾	0.0 ▾	0.0 ▾	0 ▾
43723161	helm/helm	1993	0.06 ▾	0.0 ▾	0.048 ▾	0 ▾
33884891	apache/airflow	12181	0.056 ▾	0.0 ▾	0.0 ▾	0 ▾
17165658	apache/spark	26263	0.04 ▾	0.028 ▾	0.008 ▾	1 ▾
1103607	jenkinsci/jenkins	12367	0.035 ▾	0.0 ▾	0.003 ▾	1 ▾
24195339	angular/angular	9866	0.032 ▾	0.0 ▾	0.0 ▾	0 ▾
75821432	llvm/llvm-project	58790	0.025 ▾	0.025 ▾	0.0 ▾	1 ▾
11225014	etcd-io/etcd	1495	0.024 ▾	0.0 ▾	0.0 ▾	0 ▾
20773773	bazelbuild/bazel	11491	0.022 ▾	0.003 ▾	0.001 ▾	1 ▾
70107786	vercel/next.js	28573	0.021 ▾	0.0 ▾	0.0 ▾	1 ▾
180687624	aquasecurity/trivy	3502	0.018 ▾	0.003 ▾	0.008 ▾	1 ▾
16563587	cockroachdb/cockroach	20150	0.013 ▾	0.0 ▾	0.004 ▾	1 ▾
23096959	golang/go	15262	0.012 ▾	0.001 ▾	0.005 ▾	1 ▾
45717250	tensorflow/tensorflow	36236	0.01 ▾	0.003 ▾	0.0 ▾	1 ▾
10270250	facebook/react	6882	0.006 ▾	0.0 ▾	0.0 ▾	0 ▾

17728164	hashicorp/terraform	5189	0.006 ▾	0.001 ▾	0.002 ▾	1 ▾
6838921	prometheus/prometheus	1591	0.005 ▾	0.0 ▾	0.0 ▾	0 ▾
65600975	pytorch/pytorch	21675	0.005 ▾	0.0 ▾	0.0 ▾	1 ▾
74175805	istio/istio	6308	0.005 ▾	0.0 ▾	0.001 ▾	0 ▾
7691631	moby/moby	12501	0.003 ▾	0.0 ▾	0.001 ▾	1 ▾
267054247	anchore/grype	1170	0.003 ▾	0.001 ▾	0.002 ▾	1 ▾
156018	redis/redis	1755	0.002 ▾	0.0 ▾	0.0 ▾	0 ▾
27193779	nodejs/node	47546	0.002 ▾	0.0 ▾	0.001 ▾	1 ▾
88013053	docker/cli	3984	0.002 ▾	0.0 ▾	0.001 ▾	0 ▾
724712	rust-lang/rust	58963	0.001 ▾	0.0 ▾	0.0 ▾	1 ▾
20580498	kubernetes/kubernetes	29226	0.001 ▾	0.0 ▾	0.0 ▾	0 ▾
33015583	keras-team/keras	1007	0.0 ▾	0.0 ▾	0.0 ▾	0 ▾
338633268	sigstore/sigstore	302	0.0 ▾	0.0 ▾	0.0 ▾	0 ▾